Milestone 3 Report

Smart MCQ Solver — Retrieval-Augmented Generation (RAG)

A fully explained walkthrough of the milestone-3.ipynb notebook: every question, the reasoning behind each approach, the exact code used, the notebook's output, and the final answer — written so that anyone unfamiliar with the notebook can understand what was done and why.

Source notebook: milestone-3.ipynb
Dataset: Kaggle "smart-mcq-solver-challenge"

1. Project Overview

This notebook introduces **Retrieval-Augmented Generation (RAG)** for the same multiple-choice question answering (MCQ) problem tackled in Milestones 1 and 2 — each row provides a question (**prompt**) and five candidate options (**A** through **E**), and the goal is to identify the correct option (**answer**). Rather than asking a model to answer from its own knowledge alone, RAG first retrieves relevant supporting text from an external knowledge base and injects it into the prompt before the model makes its prediction. The notebook is organized as a guided sequence of 8 questions covering vector retrieval with FAISS, reranking with a Cross-Encoder, context-window/token budgeting, the measurable benefit (and danger) of injected context, and a full end-to-end retrieve-rerank-augment-predict pipeline evaluated with the same MAP@3 metric used throughout the project.

1.1 Environment & Setup

The notebook begins by verifying the software and hardware environment it is running in, then confirms that the three competition data files are present on disk before any analysis begins.

Python	3.10.12
Platform	Linux 6.6.114.1-microsoft-standard-WSL2 (WSL2)
NumPy	2.2.6
Pandas	2.3.3
PyTorch	2.5.1+cu121
Transformers	5.13.0
Device	CUDA (NVIDIA GeForce RTX 3050 Laptop GPU, ~4.29 GB VRAM)
Retrieval stack	faiss-cpu (IndexFlatL2), sentence-transformers/all-MiniLM-L6-v2
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2
Zero-shot model	facebook/bart-large-mnli

1.2 Dataset

The dataset is read from three CSV files with pandas. A quick inspection with **.describe()**, **.shape** and **.isna().sum()** confirms the data is clean (no missing values) before any modeling begins.

Files	train.csv, test.csv, sample_submission.csv
train.csv shape	2000 rows × 8 columns
test.csv shape	500 rows × 7 columns

sample_submission.csv shape	500 rows × 2 columns
Columns	id, prompt, A, B, C, D, E, answer (test.csv omits answer)
Missing values	None — all columns fully populated across train.csv
Knowledge base (KB)	2000 documents — one per row, holding the text of that row's correct option (row 150's answer lives at KB index 150, etc.)
Vector index	faiss.IndexFlatL2 over 2000 × 384-dim MiniLM embeddings of the KB

1.3 Building the Knowledge Base & FAISS Index

Before any of the 8 questions can be answered, a required setup cell builds the milestone's retrieval infrastructure. For every one of the 2000 training rows, the text of that row's **correct** option is stored as a document in a knowledge base list **kb** — so **kb[i]** always holds the correct-answer text for row **i**. Every document is embedded into a 384-dimensional vector with **sentence-transformers/all-MiniLM-L6-v2**, and all 2000 vectors are added to a **faiss.IndexFlatL2** index — an exact (non-approximate) nearest-neighbour index that measures similarity via Euclidean (L2) distance between embeddings, letting any future query vector be compared against all 2000 documents almost instantly instead of via a slow linear scan.

2. Table of Contents

Zero-Shot Baseline & Knowledge Base Setup

Q1. Zero-Shot Classification Baseline (No Retrieval)

Vector Retrieval with FAISS (Bi-Encoder Search)

Q2. Bi-Encoder Retrieval Rank via FAISS

Reranking with a Cross-Encoder

Q3. Reranking the Top-10 with a Cross-Encoder

Context Window & Tokenization Limits

Q4. Measuring Context-Window Size in Tokens

Retrieval-Augmented Generation in Action

Q5. RAG With the True Document: Measuring the Uplift

Q6. Adversarial RAG: The Danger of Retrieving the Wrong Context

RAG Pipeline Evaluation

Q7. Measuring Retrieval Quality: Hit Rate @ k=5

Q8. The Full Retrieve → Rerank → Augment → Predict Pipeline (MAP@3)

3. Zero-Shot Baseline & Knowledge Base Setup

Question 1: Zero-Shot Classification Baseline (No Retrieval)

Run the zero-shot classifier on facebook/bart-large-mnli on the prompt for row index 150. Pass the 5 options (A–E) as candidate_labels. What is the predicted probability score assigned to the ground-truth correct option (the option in the answer column)? (Round to 3 decimal points)

Approach & Explanation

- Before touching retrieval at all, this question establishes a **zero-shot, no-context baseline**: the raw question is handed directly to **facebook/bart-large-mnli** with the five option texts as candidate labels, with nothing else injected into the prompt. This is the score every later RAG technique in the notebook (Q5, Q6, Q8) is trying to improve on.
- The zero-shot pipeline reframes each (prompt, option) pair as a natural-language entailment problem and returns a softmax probability distribution across all five candidate labels, sorted from most to least likely.
- **result["labels"].index(ans_150)** locates the position of the true correct option's text inside the returned (re-sorted) labels list, and the matching entry in **result["scores"]** is that option's model-assigned probability — regardless of whether the model actually ranked it first.
- The baseline score of 0.384 shows the model already leans towards the correct answer somewhat (well above the 0.20 that 5-way random chance would give it) purely from the question wording and general world/language knowledge, without any external context — a useful reference point for later comparison.

Code

```
row_150 = train.iloc[150]
prompt_150 = str(row_150['prompt'])
labels_150 = [str(row_150['A']), str(row_150['B']), str(row_150['C']), str(row_150['D']),
    str(row_150['E'])]
ans_150 = str(row_150[row_150['answer']])
print("Prompt:", prompt_150)
print("Labels:", labels_150)
print("Answer:", ans_150)

result = zs(prompt_150, candidate_labels=labels_150)

print(result)

score = result["scores"][result["labels"].index(ans_150)]

print(f"Ans: Score for the correct answer {round(score, 3)}")
```

Notebook Output

```
Prompt: Select the most accurate option: What is the butterfly effect, as
defined by Lorenz in his book "The Essence of Chaos"? based on the given
context.
Answer: The butterfly effect is the phenomenon that a small change in the
initial conditions of a dynamical framework can cause significant
```

```
distinctions in subsequent states, as defined by Lorenz in his book "The
Essence of Chaos."

{'sequence': '...', 'labels': [...5 reordered option texts...],
 'scores': [0.3844, 0.3787, 0.0927, 0.0786, 0.0656]}
Ans: Score for the correct answer 0.384
```

Zero-shot (no-context) probability for the correct option, row 150 = 0.384

4. Vector Retrieval with FAISS (Bi-Encoder Search)

Question 2: Bi-Encoder Retrieval Rank via FAISS

Embed the prompt for row index 150 using all-MiniLM-L6-v2. Query your FAISS index to retrieve the top k=10 most similar documents. At what exact rank (1 through 10) did FAISS place the true correct document (which is the document originally located at index 150 in the KB)?

Approach & Explanation

- The milestone's setup cell builds a **knowledge base (KB)**: for every row in train.csv, the text of that row's *correct* option is stored as one KB document, so KB index 150 holds exactly the correct-answer text for row 150. Every document is embedded with **all-MiniLM-L6-v2** into a 384-dimensional vector, and all 2000 vectors are added to a **faiss.IndexFlatL2** — a brute-force index that measures similarity via Euclidean (L2) distance between embeddings.
- To answer this question, the row-150 **prompt** (not the answer!) is embedded with the same MiniLM model and used to query the FAISS index for its **k=10** nearest neighbours — the 10 KB documents whose embeddings are closest to the prompt's embedding.
- FAISS returns the neighbours' KB indices in ranked order (closest first). Checking whether **150** appears anywhere in that returned list, and if so at what position, reveals how well a pure bi-encoder retrieval step alone can surface the correct answer's own text purely from question-to-document embedding similarity.
- The correct document lands at **rank 10** — dead last of the 10 retrieved results — which is a deliberately unflattering outcome. It demonstrates the well-known weakness of bi-encoders described in the notebook: because the question and the document are embedded completely independently (with no cross-attention between their words), the model can miss the true best match even when it is technically inside the top-k window. This motivates the reranking step in Q3.

Code

```
query = str(train.iloc[150]['prompt'])
query_embedding = model.encode([query], convert_to_numpy=True)

# search for top 10
distances, indices = index.search(query_embedding, k=10)

# Top-10 indices returned by FAISS in Q2
retrieved_indices = indices[0]
print(f"returned indices: {retrieved_indices}")
if 150 in retrieved_indices:
    rank = list(retrieved_indices).index(150) + 1
    print(f"Correct answer found at rank {rank} in the top 10 results.")
else:
    print("Correct answer not found in the top 10 results.")
```

Notebook Output

```
returned indices: [ 663 1701 1269 1532  576  847 1693 1906  168  150]
```

```
Correct answer found at rank 10 in the top 10 results.
```

FAISS bi-encoder rank of the true document (KB index 150) among its own top-10 = rank 10

5. Reranking with a Cross-Encoder

Question 3: Reranking the Top-10 with a Cross-Encoder

Take the top 10 documents retrieved by FAISS in the previous question. Load cross-encoder/ms-marco-MiniLM-L-6-v2. Score the prompt against these 10 documents and sort them by the cross-encoder's score. At what exact rank (1 through 10) does the Cross-Encoder place the true correct document?

Approach & Explanation

- Unlike a bi-encoder (which embeds the query and each document completely separately, then compares vectors), a **Cross-Encoder** feeds the **(question, document)** pair into the transformer together, in a single forward pass, so full attention can directly compare every word of the question against every word of the document. This is far more accurate but also far more computationally expensive per pair — which is exactly why it's only used to rerank a small shortlist (here, the 10 candidates FAISS already narrowed things down to) rather than searching the whole KB directly.
- **cross-encoder/ms-marco-MiniLM-L-6-v2** is loaded and given 10 **(prompt_150, document)** pairs — one for every document FAISS returned in Q2 — and outputs a raw relevance score for each pair via **cross_encoder.predict(pairs)**.
- Zipping together each KB index, its document text, and its cross-encoder score, then sorting that combined list by score (highest first), produces a brand-new ranking of the same 10 candidates — this time based on deep semantic relevance rather than raw embedding distance.
- Under this reranking, the true document (KB index 150) — which fell all the way to bi-encoder rank 10 in Q2 — jumps to **rank 1** with the highest cross-encoder score (4.7585). This is a striking, concrete illustration of why a two-stage retrieve-then-rerank pipeline is standard practice in modern RAG systems: the fast bi-encoder narrows a huge corpus down to a manageable shortlist, and the slower but far more accurate cross-encoder then puts the genuinely best match on top.

Code

```
from sentence_transformers import CrossEncoder

cross_encoder = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
# Retrieve corresponding documents
docs_10 = [kb[i] for i in retrieved_indices] #Get the top 10 chunks

# Create (question, document) pairs
pairs = [[prompt_150, doc] for doc in docs_10] #Create prompt-context pairs

# Compute Cross-Encoder scores
ce_scores = cross_encoder.predict(pairs) # Get the score of each pair

print("Cross-Encoder scores:", ce_scores)

# combine everything into a list of tuples for easier analysis
results = list(zip(retrieved_indices, docs_10, ce_scores))
# Sort by score (highest first)
```

```
results = sorted(results, key=lambda x: x[2], reverse=True)

# print ranking
for rank, (index, doc, score) in enumerate(results, start=1):
    print(f"Rank {rank}: KB Index {index}, Score {score:.4f}")
    #print(f"Document: {doc}\n")

correct_doc_index = 150

for rank, (idx, doc, score) in enumerate(results, start=1):
    if idx == correct_doc_index:
        print(f"Correct answer found at rank {rank} with score {score:.4f}")
        break
```

Notebook Output

```
Cross-Encoder scores: [4.6602297 4.6602297 4.737521  4.737521  4.687044
4.752575  4.752575  4.752575  4.7072463 4.7585096]

Rank 1: KB Index 150, Score 4.7585
Rank 2: KB Index 847, Score 4.7526
Rank 3: KB Index 1693, Score 4.7526
Rank 4: KB Index 1906, Score 4.7526
Rank 5: KB Index 1269, Score 4.7375
Rank 6: KB Index 1532, Score 4.7375
Rank 7: KB Index 168, Score 4.7072
Rank 8: KB Index 576, Score 4.6870
Rank 9: KB Index 663, Score 4.6602
Rank 10: KB Index 1701, Score 4.6602
Correct answer found at rank 1 with score 4.7585
```

Cross-Encoder rerank rank of the true document (KB index 150) = rank 1 (up from rank 10)

6. Context Window & Tokenization Limits

Question 4: Measuring Context-Window Size in Tokens

Retrieve the top k=5 documents for the prompt at row index 42. Concatenate them with a single space between each. Create a string: "Context: [concatenated_docs] Question: [prompt]". Tokenize this string using the bert-base-uncased tokenizer (without truncation). Exactly how many total tokens does this generate?

Approach & Explanation

- Real LLMs and encoders have a fixed maximum context window (BERT-base, for instance, tops out at 512 tokens), so it's essential to know exactly how many tokens a given amount of retrieved context actually consumes before deciding how many chunks (**k**) is practical to retrieve.
- The row-42 prompt is embedded and FAISS is queried for its **top k=5** nearest KB documents (the same bi-encoder retrieval mechanic as Q2, just with a smaller k and a different query row).
- The five retrieved documents are joined into one string with **" ".join(docs_5)**, then wrapped into the standard RAG-prompt template used throughout this milestone: **"Context: {concatenated_docs} Question: {prompt}"**.
- Tokenizing that full string with the **bert-base-uncased** tokenizer and explicitly passing **truncation=False** ensures every token is counted, even if it would exceed BERT's normal 512-token limit — this reveals the true, un-clipped size of the context.
- The result, 216 tokens, comfortably fits within BERT's 512-token limit for k=5, but it also shows how quickly the tokens add up — retrieving many more chunks (or much longer documents) could easily blow past the model's context window and force silent truncation, which is exactly the kind of tradeoff a real RAG system has to tune (as the notebook's introduction puts it, "the physical limits of context windows").

Code

```
# "Flat" means store all vectors exactly as they are (no compression or approximation).
# "L2" means similarity is measured using the Euclidean (L2) distance
# Smaller distance means more similar vectors.
index = faiss.IndexFlatL2(kb_embeddings.shape[1])
index.add(kb_embeddings)

# prompt at row 42
row_42 = train.iloc[42]
prompt_42 = str(row_42['prompt'])

# embedding the prompt
query_embedding_42 = model.encode([prompt_42])

# Retrive top 5 documents
distance, indices = index.search(query_embedding_42, k=5)

retrieved_indices = indices[0]
docs_5 = [kb[i] for i in retrieved_indices]
# Concatenate with a single space
```

```
concatenated_docs = " ".join(docs_5)

# Build the input string
text = f"Context: {concatenated_docs} Question: {prompt_42}"

# Tokenize without truncation
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

encoding = tokenizer(text, truncation=False)

# Count total tokens
num_tokens = len(encoding["input_ids"])

print("Total tokens:", num_tokens)
```

Notebook Output

```
Total tokens: 216
```

Total BERT token count for "Context: [top-5 docs] Question: [prompt]" (row 42) = 216 tokens

7. Retrieval-Augmented Generation in Action

Question 5: RAG With the True Document: Measuring the Uplift

Retrieve the exact true document for row index 150 from your KB. Create a RAG string: "Context: [true_document] Question: [prompt]". Run the same zero-shot classification from Question 1 on this augmented string. What is the new predicted probability score of the ground-truth correct option? (Round to 3 decimal places).

Approach & Explanation

- This is the central proof-of-concept for the entire milestone: instead of relying on retrieval to find the right document (as Q2/Q3 did), the true correct-answer text is looked up directly at **kb[150]** (an oracle / best-case scenario) and injected as context, to isolate and measure exactly how much a perfect retrieval step can help.
- The RAG-formatted string follows the same template used throughout the notebook: **"Context: {true_document} Question: {prompt_150}"** — the true document is prepended before the original question.
- This augmented string, rather than the bare prompt, is fed into the exact same zero-shot classification pipeline and candidate labels used in Q1, so the two scores are directly comparable apples-to-apples.
- The correct option's probability jumps from Q1's context-free baseline of **0.384** to **0.989** — essentially full model confidence. This dramatic uplift is the core argument for Retrieval-Augmented Generation: giving a model the right supporting evidence, even without any fine-tuning, can turn a mediocre guess into a near-certain correct answer.

Code

```
row_150 = train.iloc[150]

prompt_150 = str(row_150['prompt'])

true_document = kb[150]

rag_input = f"Context: {true_document} Question: {prompt_150}"

labels = [
    str(row_150['A']),
    str(row_150['B']),
    str(row_150['C']),
    str(row_150['D']),
    str(row_150['E'])
]

# run the zero shot classification pipeline
result = zs(
    rag_input,
    candidate_labels=labels
)
correct_answer = str(row_150[row_150['answer']])
```

```
score = result["scores"][result["labels"].index(correct_answer)]

print(f"Score: {round(score, 3)}")
```

Notebook Output

```
Score: 0.989
```

Zero-shot probability for the correct option WITH the true document as context (row 150) = 0.989 (up from 0.384 with no context)

Question 6: Adversarial RAG: The Danger of Retrieving the Wrong Context

What happens if your vector database retrieves the wrong information? Take the prompt for row index 150. Manually force the context to be the document located at KB index 999 (a completely unrelated fact). Run the zero-shot classifier on this "Adversarial RAG" string. What is the probability of the correct option now? (Round to 3 decimal places).

Approach & Explanation

- This question deliberately breaks the pipeline to illustrate RAG's central risk, summarized in the notebook as "Garbage In, Garbage Out": a language model tends to place a great deal of trust in whatever context it is given, so if retrieval goes wrong and surfaces an irrelevant document, the model can be misled into confidently reasoning from bad evidence rather than falling back on its own internal knowledge.
- Instead of retrieving anything, the context is *manually forced* to be **kb[999]** — a KB document that is the correct answer to a completely different, unrelated question, deliberately simulating a retrieval failure.
- The exact same RAG-string template and zero-shot classification call from Q1/Q5 are reused, so this is a clean, controlled comparison against both the no-context baseline (Q1: 0.384) and the perfect-context result (Q5: 0.989).
- The correct answer's probability drops to **0.529** — still noticeably higher than the pure no-context baseline of 0.384 (since the question wording alone still carries some signal), but far below the near-certainty (0.989) achieved with the true document. This quantifies the danger described in the concept note above: bad or irrelevant retrieved context measurably degrades the model's confidence in the correct answer, even if it doesn't necessarily flip the prediction outright in this particular example — which is precisely why reranking (Q3) and careful control over how many/which chunks are retrieved matter so much in a real RAG system.

Code

```
row_150 = train.iloc[150]

prompt_150 = str(row_150["prompt"])
correct_answer = str(row_150[row_150["answer"]])

# Assuming 999 is a wrong document index for demonstration
```

```
wrong_document = kb[999]

adversarial_rag = (
    f"Context: {wrong_document} Question: {prompt_150}"
)

labels = [
    str(row_150["A"]),
    str(row_150["B"]),
    str(row_150["C"]),
    str(row_150["D"]),
    str(row_150["E"])
]

result_adversarial = zs(
    adversarial_rag,
    candidate_labels=labels
)

result_adversarial_score =
    result_adversarial["scores"][result_adversarial["labels"].index(correct_answer)]

print(f"Probability score for the correct answer with adversarial context:
    {round(result_adversarial_score, 3)}")
```

Notebook Output

```
Probability score for the correct answer with adversarial context: 0.529
```

Zero-shot probability for the correct option WITH an unrelated/adversarial document as context (row 150) = 0.529 (vs. 0.384 no-context and 0.989 true-context)

8. RAG Pipeline Evaluation

Question 7: Measuring Retrieval Quality: Hit Rate @ k=5

For the first 100 rows of train.csv (indices 0–99), retrieve the top k=5 documents for each prompt. If the exact string of the row's correct option is found inside any of those 5 retrieved documents, it counts as a hit. What is the exact Hit Rate percentage (0 to 100) for these 100 rows? (Round to 1 decimal place).

Approach & Explanation

- **Hit Rate** is a standard retrieval-quality metric, independent of any downstream generation step: it simply asks, across many queries, what fraction of the time the retriever's returned set actually contains the fact needed to answer the question at all. It measures the retriever in isolation, before any reranking or generation can help or hurt.
- For each of the first 100 rows, the prompt is embedded and FAISS returns its **top k=5** nearest KB documents — the same bi-encoder retrieval step used elsewhere in the notebook, just repeated at scale across many queries this time.
- A "hit" is counted whenever the exact string of that row's correct-answer option text is present among those five retrieved documents (a direct string membership check, **correct_answer in docs_5**) — note this is a stricter, exact-match criterion rather than a semantic-similarity threshold.
- Out of 100 queries, 73 produced a hit, giving a Hit Rate of **73.0%**. This means that roughly 3 out of every 4 questions had their correct answer's exact text somewhere within the top-5 retrieved documents — a solid but clearly imperfect retrieval quality, consistent with the earlier observation in Q2 that even when the correct document IS retrieved, it isn't necessarily ranked first (which is exactly the gap that reranking, per Q3, is designed to close).

Code

```
hits = 0

for i in range(100):
    # current row
    row = train.iloc[i]

    # prompt
    prompt = str(row["prompt"])

    # correct answer
    correct_answer = str(row[row["answer"]])

    # embed the prompt
    query_embedding = model.encode([prompt])

    # retrieve top 5 documents
    distance, indices = index.search(query_embedding, k=5)

    retrieved_indices = indices[0]
    # retrieve the corresponding documents
```

```
    docs_5 = [kb[i] for i in retrieved_indices]

    if correct_answer in docs_5:
        hits += 1

hit_rate = (hits / 100) * 100

print(f"Number of hits in the first 100 prompts: {hits}")
print(f"Hit rate for the first 100 prompts: {hit_rate:.2f}%")
```

Notebook Output

```
Number of hits in the first 100 prompts: 73
Hit rate for the first 100 prompts: 73.00%
```

Retriever Hit Rate @ k=5 (first 100 rows) = 73.0%

Question 8: The Full Retrieve → Rerank → Augment → Predict Pipeline (MAP@3)

Build a loop that processes the first 20 rows (indices 0–19) of train.csv. For each row: Retrieve the top k=5 FAISS documents; Rerank them with the ms-marco-MiniLM-L-6-v2 Cross-Encoder and keep the single best document; Augment the prompt with that document ("Context: [best_document] Question: [prompt]"); Predict with the facebook/bart-large-mnli zero-shot classifier over the 5 options; take the top-3 predicted letters and compute MAP@3 for that row. What is the final average MAP@3 across these 20 rows? (Round to 3 decimal places).

Approach & Explanation

- This question assembles every technique from the milestone into a single, complete, state-of-the-art RAG pipeline, run end-to-end on 20 real rows and scored with the same MAP@3 metric used to evaluate the classical baselines in Milestone 1 (majority-class: 0.4213) and the semantic-embedding pipeline in Milestone 2 (MiniLM MAP@3: 0.4231) — allowing a direct, apples-to-apples comparison across all three milestones.
- **Step 1 — Retrieve:** the prompt is embedded with MiniLM and FAISS returns its **top k=5** nearest KB documents (same as Q4/Q7).
- **Step 2 — Rerank:** all 5 retrieved documents are paired with the prompt and scored by the **ms-marco-MiniLM-L-6-v2 Cross-Encoder** (same mechanism as Q3), and **np.argmax(ce_scores)** picks out the single highest-scoring document as the best possible context — rather than passing all 5 chunks through, only the cross-encoder's #1 pick is kept, keeping the context window small and focused.
- **Step 3 — Augment:** that single best document is inserted into the same **"Context: {best_doc} Question: {prompt}"** template used throughout the notebook.
- **Step 4 — Predict:** the augmented string is scored by **facebook/bart-large-mnli** zero-shot classification against the row's 5 option texts as candidate labels, exactly as in Q1/Q5/Q6.

- **Step 5 — Score:** the returned labels (already sorted by probability, highest first) are mapped back to their corresponding option letters, the top 3 are kept as the row's ranked prediction, and **map_at_3()** (the same 1/(rank+1) scoring rule used in Milestones 1 and 2) is applied — then averaged across all 20 rows.
- The final score, **MAP@3 = 0.975**, is dramatically higher than either earlier milestone's baseline. This is strong end-to-end evidence for the value of the full two-stage retrieve-and-rerank RAG pipeline: narrowing candidates with a fast bi-encoder, sharpening the selection with an accurate cross-encoder, and then letting a zero-shot classifier reason over one well-chosen, highly relevant piece of context comes very close to perfect performance on this small evaluation slice.

Code

```
def map_at_3(actual, predicted):
    """
    actual: correct option letter, e.g. 'C'
    predicted: list of top-3 predicted option letters
    """
    for i, p in enumerate(predicted):
        if p == actual:
            return 1.0 / (i + 1)
    return 0.0

scores = []
for i in range(20):
    row = train.iloc[i]
    prompt = str(row["prompt"])

    # Retrieve (FAISS)
    query_embedding = model.encode([prompt])
    distance, indices = index.search(query_embedding, k=5)
    retrieved_indices = indices[0]
    docs_5 = [kb[i] for i in retrieved_indices]

    # Rerank (Cross-Encoder)
    pairs = [[prompt, doc] for doc in docs_5]
    ce_scores = cross_encoder.predict(pairs)
    best_doc = docs_5[np.argmax(ce_scores)]

    # RAG (Zero-Shot Classification)
    rag_prompt = f"Context: {best_doc} Question: {prompt}"
    labels = [
        str(row["A"]),
        str(row["B"]),
        str(row["C"]),
        str(row["D"]),
        str(row["E"])
    ]

    # Zero-Shot Classification
    result  = zs(rag_prompt, candidate_labels=labels)

    correct_option = row["answer"] # correct option from the row

    predicted_options = []
    for option_text in result["labels"][:3]: # Get top 3 predicted options
        for option in ["A", "B", "C", "D", "E"]:
            if str(row[option]) == option_text:
                predicted_options.append(option)
                break
```

```
    # MAP@3
    score = map_at_3(correct_option, predicted_options)
    scores.append(score)

# Final MAP@3 score
final_map_at_3 = np.mean(scores)
print(f"Final MAP@3 score for the first 20 prompts: {round(final_map_at_3, 3)}")
```

Notebook Output

```
Final MAP@3 score for the first 20 prompts: 0.975
```

Full Retrieve→Rerank→Augment→Predict RAG pipeline MAP@3 (first 20 rows) = 0.975

9. Summary of Answers

The table below collects every question's final answer in one place for quick reference.

Q#	Answer
Q1	Zero-shot (no-context) probability for the correct option, row 150 = 0.384
Q2	FAISS bi-encoder rank of the true document (KB index 150) among its own top-10 = rank 10
Q3	Cross-Encoder rerank rank of the true document (KB index 150) = rank 1 (up from rank 10)
Q4	Total BERT token count for "Context: [top-5 docs] Question: [prompt]" (row 42) = 216 tokens
Q5	Zero-shot probability for the correct option WITH the true document as context (row 150) = 0.989 (up from 0.384 with no context)
Q6	Zero-shot probability for the correct option WITH an unrelated/adversarial document as context (row 150) = 0.529 (vs. 0.384 no-context and 0.989 true-context)
Q7	Retriever Hit Rate @ k=5 (first 100 rows) = 73.0%
Q8	Full Retrieve→Rerank→Augment→Predict RAG pipeline MAP@3 (first 20 rows) = 0.975

Key Takeaways

- With no retrieved context at all, zero-shot classification already scores 0.384 on the correct option for row 150 — well above 5-way random chance (0.20), purely from question wording and the model's own language knowledge.
- A fast bi-encoder (FAISS + MiniLM) is not always precise: for row 150, the true document was retrieved but landed at rank 10 of 10 — technically "found" but practically useless without reranking.
- A Cross-Encoder reranking step fixes exactly this weakness by scoring the question and each candidate document together with full attention, promoting the true document from rank 10 all the way to rank 1 for the same query.
- Context size has real, measurable token cost: concatenating just the top-5 retrieved documents with a question already consumes 216 BERT tokens, underscoring why retrieval systems must budget carefully against a model's context window.
- Injecting the correct supporting document as context lifts the correct answer's zero-shot probability from 0.384 to 0.989, while injecting an unrelated, wrong document instead only reaches 0.529 — concrete, quantified evidence for both the power and the danger ("garbage in, garbage out") of RAG.

- Retrieval quality itself, measured independently of generation, achieves a 73.0% Hit Rate @ k=5 across 100 rows — meaning roughly 1 in 4 queries doesn't even have the correct answer's exact text anywhere in its top-5 retrieved documents.
- The complete Retrieve → Rerank → Augment → Predict pipeline achieves MAP@3 = 0.975 on a 20-row sample — dramatically higher than the majority-class baseline (0.4213, Milestone 1), the plain TF-IDF pipeline (0.2962, Milestone 1), and the semantic MiniLM-only pipeline (0.4231, Milestone 2) — the strongest evidence yet that combining retrieval, reranking, and zero-shot reasoning substantially outperforms any single technique alone.